

1. Regular Expression:

$$(a/b)^* a (a/b) (a/b)$$

X Good

86
100

2. Illustrate the predictive parser for the following grammar.

$$S \rightarrow (L) / a$$

$$L \rightarrow L, S / S$$

Given Production

$$S \rightarrow (L)$$

$$S \rightarrow a$$

$$L \rightarrow L'$$

$$L' \rightarrow S / S$$

$$\text{first}(S) = \{ (, a \} \quad \text{follow}(S) = \{ \epsilon,) \}$$

$$\text{first}(S) = \{ (, a \}$$

$$\text{follow}(S) = \{ \epsilon,) \}$$

$$\text{first}(L) = \{ (, S \}$$

$$\text{follow}(L) = \{ \epsilon,) \}$$

$$\text{first}(L') = \{ S \}$$

$$\text{follow}(L') = \{ a, S, C, \epsilon \}$$

$$\{ a, S, C, \epsilon \}$$

Non-terminal	a	C	ϵ	S
S	$S \rightarrow a$	$S \rightarrow (L)$		
L				$L \rightarrow L'$
L'				$L' \rightarrow S$

3. Analyse. is it possible by modifying the grammar
 $S \rightarrow SS+ / SS^* / a$ String: "aa+a"

Production

$$S \rightarrow SS +$$

$$S \rightarrow SS^*$$

$$S \rightarrow a$$

String "aa+a"

Stack	String	Output.
\$	aa+a\$	$S \rightarrow SS +$
\$+SS	aa+a\$	$S \rightarrow a$
\$+aa	aa+a\$	
\$+	aa+a\$	
\$	aa+a\$	
\$a	aa+a\$	
\$	aa+a\$	

4. Eliminate left recursion from grammar

$$S \rightarrow Sa / Sb / c$$

$$S \rightarrow Sa$$

$$S \rightarrow Sb$$

$$S \rightarrow c$$

$$\text{first}(S) = aSs' / bSs' / c$$

$$\text{follow}(S) = \epsilon$$

$$\text{Left Recursion} = aSs' / bSs' / c$$

The following grammar contains common prefixes, making it unsuitable for predictive parsing.

$$A \rightarrow abcD / abcE / f$$

Apply left factoring to remove the common prefixes and rewrite the given production.

$$A \rightarrow abcD / abcE / f$$

$\propto BB_1, \propto BB_2$

$$A \rightarrow abc / abc / \epsilon$$

$$A \rightarrow abc / \epsilon$$

6. Consider the following statement

$$\text{Degree} - f = \text{Degree} - C * 1.8 + 32$$

Phases of Compiler

Phase 1 :- Lexical Analyzer.

$$\text{Degree} - f = \text{Degree} - C * 1.8 + 32$$

$$id = id * 1.8 + 32$$

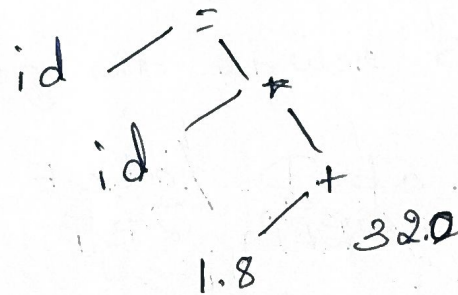
$$\langle id \rangle \langle = \rangle \langle id \rangle \langle * \rangle \langle 1.8 \rangle \langle + \rangle \langle 32 \rangle$$

Identifying the keywords and identifiers

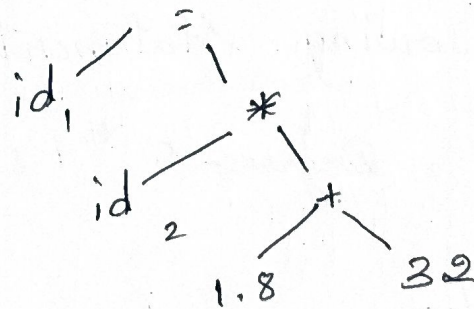
Phase 3:- ~~Syntax~~ analyzer
Sematic

$$\text{Degree}_f = \text{Degree}_c * 1.8 + 32$$

$$id = id * 1.8 + 32$$



Phase 2:- Syntax Analyzer



Phase 4:- Intermediate code generator

$$\text{temp1} = 32$$

$$\text{temp2} = 1.8 + 32$$

$$\text{temp3} = id_2 * \text{temp2}$$

$$\text{temp4} = id_1$$

$$id_4 = \text{temp3}$$

Phase 5 :- Code Optimizer.

temp 1 = 1.8 + 32

temp 2 = id₂ * temp 1

id₁ = temp 2

Phase 6 :- Code generator.

~~ADD R₁, id, temp 1~~
~~MUL R₁, id₂~~

STORE R₁, 32

STORE R₂, 1.8

ADD R₃, R₁, R₂

STORE R₄, id₂

~~MUL R₅, R₃, R₄~~

STORE R₆, R₅

1. Consider the following C code fragment.

```
sum = 0;  
for (i = 1; i < 5; i++)  
    sum = sum + i;
```

sum = sum + i;

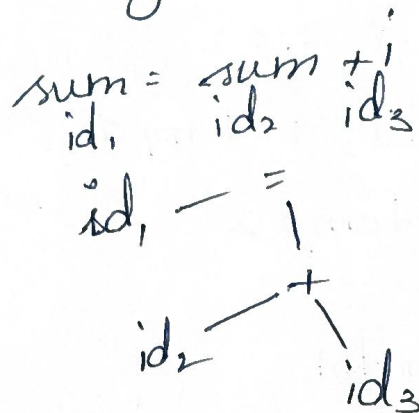
Phase 1 :- Lexical phase

~~sum = sum + i;~~

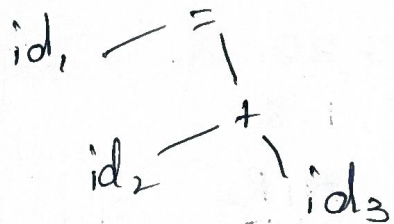
~~id = id + id~~

<id> <=> <id> <+> <id>

Phase 2 :- Syntax analyzer



Phase 3 :- Semantic analyzer



Phase 4 :- Intermediate code generator

temp1 = id₃
temp2 = id₂ + temp1
~~temp3~~ id₁ = temp2

Phase 5 :- Code optimizer

temp1 = id₂ + id₃
id₁ = temp1

Phase 6 :- Code generator

STORE R₁, id₃
STORE R₂, id₂
STORE R₃, id₁
ADD R₄, R₁, R₂
STORE R₅, R₄

Consider the grammar

$S \rightarrow aABb$, $A \rightarrow c/\epsilon$, $B \rightarrow d/\epsilon$

$S \rightarrow aABb$

$A \rightarrow c/\epsilon$

$B \rightarrow d/\epsilon$

String :- acdb.

$\text{first}(S) = \{a, b\}$

$\text{first}(A) = \{c, \epsilon\}$

$\text{first}(B) = \{d, \epsilon\}$

$\text{follow}(S) = \{\phi\}$

$\text{follow}(A) = \{a, c, \phi\}$

$\text{follow}(B) = \{a, cd, \phi\}$

Predictive table.

Non-terminals	a	b	c	d	ϕ	$\#$
S					$S \rightarrow aABb$	
A	$A \rightarrow \epsilon$		$A \rightarrow c$		$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
B	$B \rightarrow \epsilon$		$B \rightarrow \epsilon$	$B \rightarrow d$	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$

9. Write a simple C program to recognize and count identifiers in the given output.

%. %.

int age;

if (age > 18)

print ("eligible to vote")

else if (age < 18)

Print ("Not eligible to vote")

else

Print ("Invalid")

%%

String : Eligible = age > 18
 $id_1 = id_2 > 18$

Identifiers are id_1 & id_2

Operators are = , >

87 String ~~ab~~ acdb

$S \rightarrow aABb$

$A \rightarrow c/\epsilon$

$B \rightarrow d/\epsilon$

Stack	String	Output
ϕ	acdb ϕ	$S \rightarrow aABb$
$\phi aABb$	acdb ϕ	
ϕABb	cdb ϕ	$A \rightarrow c$
$\phi ABbc$	cdb ϕ	$B \rightarrow d$
ϕbd	db ϕ	
ϕb	b ϕ	
ϕ	ϕ	

Write the rules for left recursion and left factoring.

If we have same Terminals we have to perform left recursion

If we have common Terminal we have to perform left factoring

Consider Grammar: $E \rightarrow E+T / T$ $E' \rightarrow +TE'$
 $T \rightarrow T * f / f$ $E' \rightarrow T / \epsilon$
 $f \rightarrow (E) / id$ $T \rightarrow * f T'$
 $T' \rightarrow f / \epsilon$
 $f \rightarrow (E) / id$

Eliminate first and follow.

first $(E) = \{ (, id \}$

first $(E') = \{ +, \epsilon \}$

first $(T) = \{ (, id \}$

first $(T') = \{ *, \epsilon \}$

first $(f) = \{ (, id \}$

follow $(E) = \{ \$,) \}$

follow $(E') = \{ \$,) \}$

follow $(T) = \{ \$,), + \}$

follow $(T') = \{ \$,), *, + \}$

follow $(f) = \{ \$,), *, + \}$

4

11. What are the actions performed by a reduce parser? Explain each action with a suitable example:-

* Example:- $E + E * E$ we have to reduce this to $\$$

* There are productions given $E + E = E$, $E * E = E$

* If non-terminal is greater than the operator so non-terminal can be shifted first

* rule is non-terminal id $> + > * > / > \epsilon$

* we can reduce the string when the condition gets satisfied that $E + E \rightarrow E$, $E \rightarrow a$

~~the end~~

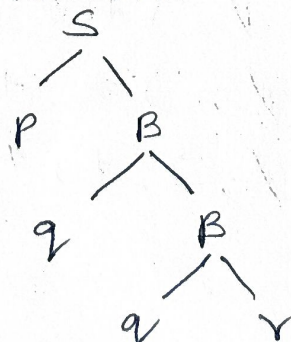
12. Consider the following grammar:-

$$S \rightarrow p B$$

$$B \rightarrow q B / r$$

string $pqq r$.

Left most derivation.



Shift Reduce Parsing.

$$S \rightarrow AB$$

$$A \rightarrow aA/a$$

$$B \rightarrow cB/c$$

string:- abcc

	Input	Output
\$	abcc\$	shift
\$a	bcc\$	Reduce $A \rightarrow a$
\$A	bcc\$	shift
\$Ab	cc\$	shift
\$Abc	c\$	shift
\$Abcc	\$	Reduce $B \rightarrow c$
\$AbccB	\$	Reduce
\$AbB	\$	Reduce $S \rightarrow AB$
\$bAB	\$	Not acceptable.
\$bAB\$	\$	

15. For the following grammar, verify whether the string abbc is valid using shift reduce parsing.

$$S \rightarrow aAB$$

$$A \rightarrow bA/b$$

$$B \rightarrow c$$

Input	Output
\$	shift
abbc \$	shift
bbc \$	Reduce $S \rightarrow aAB$
bc \$	Reduce $A \rightarrow bA/b$
bc \$	Reduce $B \rightarrow c$
c \$	shift
\$	shift
\$	Reduce $c \rightarrow BA \rightarrow bA$
\$	Reduce.
\$	$S \rightarrow aAB$
\$	Accept.

16. Consider the following grammar.

$$S' \rightarrow S$$

$$S \rightarrow aAB$$

$$A \rightarrow bA/c$$

$$B \rightarrow d.$$

SLR Parser.

LR(0) =

$$\begin{aligned}
 &S' \rightarrow \cdot S \\
 &S \rightarrow \cdot aAB \\
 &A \rightarrow \cdot bA/c \\
 &B \rightarrow \cdot d
 \end{aligned}$$

$$\underline{\text{Goto } (I_0, s) = I_1}$$

$$s' \rightarrow s. \checkmark$$

$$\underline{\text{Goto } (I_0, b) = I_3}$$

$$A \rightarrow b. A / c$$

$$\underline{\text{Goto } (I_0, a) = I_2}$$

$$S \rightarrow a. AB$$

$$A \rightarrow . b A / c$$

$$\underline{\text{Goto } (I_0, d) = I_4}$$

$$B \rightarrow d. \checkmark$$

$$\underline{\text{Goto } (I_2, A) = I_6}$$

$$S \rightarrow a A. B$$

$$B \rightarrow . d$$

$$A \rightarrow b A. / c$$

$$\underline{\text{Goto } (I_0, c) = I_5}$$

$$A \rightarrow c.$$

$$\underline{\text{Goto } (I_6, B) = I_7}$$

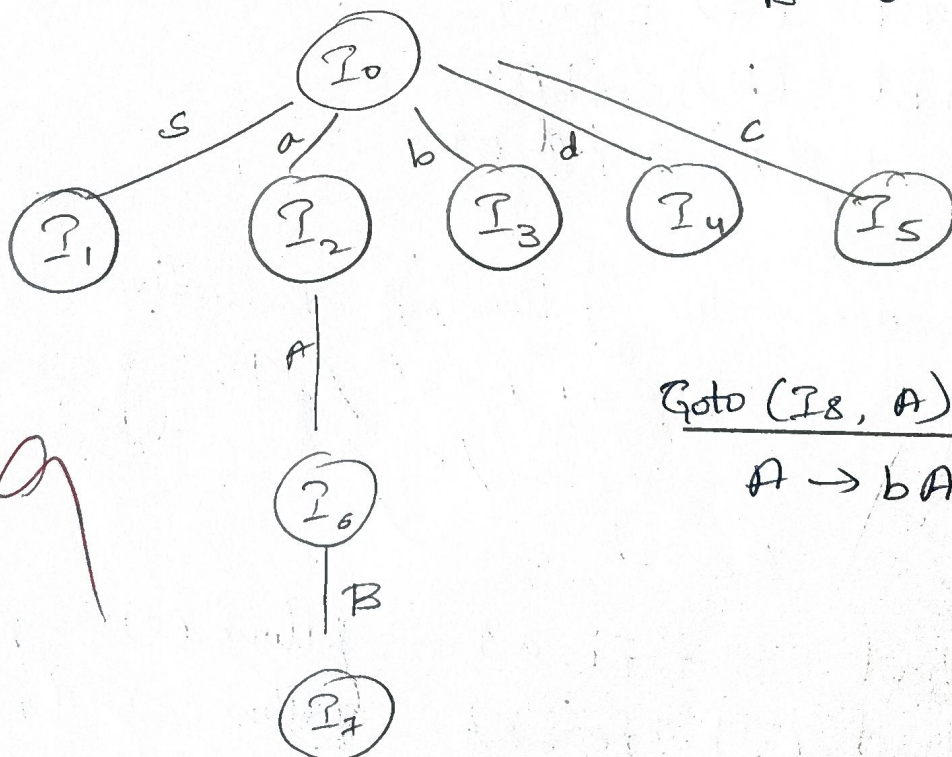
$$S \rightarrow a A B.$$

$$\underline{\text{Goto } (I_9, b) = I_8}$$

$$A \rightarrow b. A$$

$$\underline{\text{Goto } (I_6, d) = I_9}$$

$$B \rightarrow d.$$



$$\underline{\text{Goto } (I_8, A) = I_6}$$

$$A \rightarrow b A.$$

	a	b	c	d	S	A
Q	S ₂	S ₃	S ₅	S ₄	1	
1					acc	
2	r	S ₈				
3						
4				r ₃		
5						
6	r	r ₂	r ₂	S ₉		
7	r ₁					

first (S) = {a}

first (A) = {b, ε}

first (B) = {d, ε}

17. Consider the following Predictive table.

$S \rightarrow iEtSS' / a$

$S' \rightarrow eS / \epsilon$

$E \rightarrow b$

first (S) = {i, a}

first (S') = {e, ε}

first (E) = {b}

follow (S) = {\$, ε}

follow (S') = {\$, ε}

follow (E) = {\$, ε}

Non-terminals	c	E	ϕ
S	$S \rightarrow iEtss'$		$S \rightarrow iEtss'$
S'	$S' \rightarrow \epsilon$		$S' \rightarrow \epsilon$
E	$E \rightarrow b$		$E \rightarrow b$

Predictive Parsing

#

LL (1) Parsing table.

$$S \rightarrow i s \# / \# A B C$$

$$A \rightarrow a / b b D$$

$$B \rightarrow a / \epsilon$$

$$C \rightarrow b / \epsilon$$

$$D \rightarrow c / \epsilon$$

$$\text{first}(S) = \{ i, \# \}$$

$$\text{first}(A) = \{ a, b \}$$

$$\text{first}(B) = \{ a, \epsilon \}$$

$$\text{first}(C) = \{ b, \epsilon \}$$

$$\text{first}(D) = \{ c, \epsilon \}$$

$$\text{follow}(S) = \{ \phi, \# \}$$

$$\text{follow}(A) = \{ a, b, \phi, \# \}$$

$$\text{follow}(B) = \{ a, b, \phi, \# \}$$

$$\text{follow}(C) = \{ a, b, \phi, \# \}$$

$$\text{follow}(D) = \{ a, b, c, \phi, \# \}$$

18. Do left factoring

$$A \rightarrow aAB / aA / a$$

$$B \rightarrow bB / b$$

$$* A \rightarrow aAB / aA / a$$

$$A \rightarrow a / a / a$$

$$A \rightarrow a$$

$$* B \rightarrow bB / b$$

$$B \rightarrow b / b$$

$$B \rightarrow b$$

19. Construct the LL(1) Parsing table.

$$S \rightarrow ABC$$

$$A \rightarrow a / \epsilon$$

$$B \rightarrow r / \epsilon$$

$$C \rightarrow b / \epsilon$$

$$\text{first}(S) = \{a, r, b\}$$

$$\text{first}(A) = \{a, \epsilon\}$$

$$\text{first}(B) = \{r, \epsilon\}$$

$$\text{first}(C) = \{b, \epsilon\}$$

$$\text{follow}(S) = \{a, r, b, \$\}$$

$$\text{follow}(A) = \{a, r, b, \$\}$$

$$\text{follow}(B) = \{a, r, b, \$\}$$

$$\text{follow}(C) = \{a, r, b, \$\}$$